| | | |
|---|---|---|
| **Kingdom of Saudi Arabia**<br>**Ministry of Education**<br>**University of Jeddah**<br>**College of Science and Computer Engineering** | جامعة جدة<br>University of Jeddah | **المملكة العربية السعودية**<br>**وزارة التعليم**<br>**جامعة جدّة**<br>**كلية علوم وهندسة الحاسب** |

# CCCY 322: Software Security

## Race Condition Vulnerability Lab

**Shahad Alharbi - 22106G7**

## Turning Off Countermeasures

First, I began with Sudo sysctl -w fs.protected_symlinks=0 command. That is used to turn off the kernel parameter that defends a Linux system from race condition attacks using symbolic links. An administrator creates the possibility of exploitation by running this command with superuser rights, which would allow an attacker to alter symbolic links and obtain unauthorized access to private files or folders.

```
seed@ShahadAlharbi:~/.../Lab6$sudo sysctl -w fs.protected_symlinks=0
fs.protected_symlinks = 0
```

**A Vulnerable Program vulp.c:** A Set-UID executable, the vulnerable application (vulp.c) enables users to append input to a temporary file (/tmp/XYZ). The access() method is used to verify user permissions, but there is a race condition between this check and the fopen() call. An attacker can exploit this by creating a symbolic link from /tmp/XYZ to a sensitive file (like /etc/passwd), potentially allowing unauthorized access or privilege escalation.

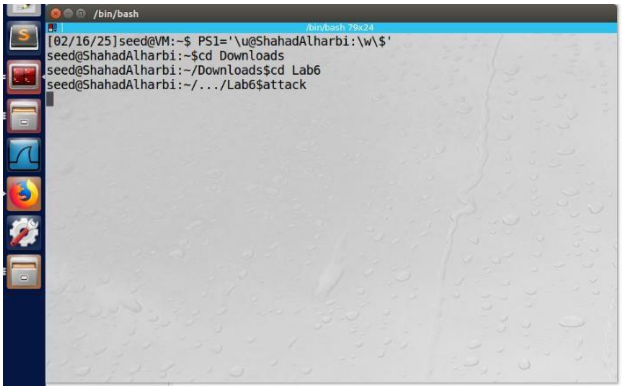
We need to compile the Vulnerable program by executing gcc vulp.c -o vulp command compiles vulp.c into an executable called vulp using the C compiler GCC.

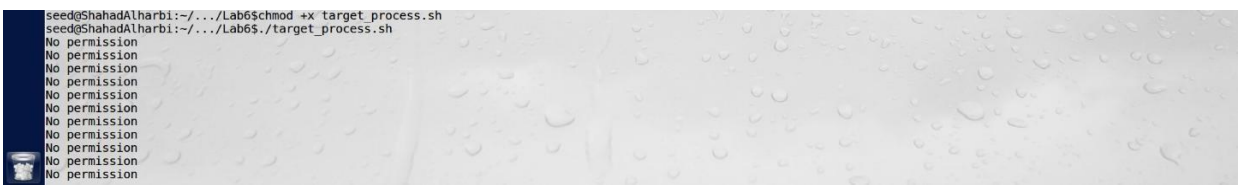
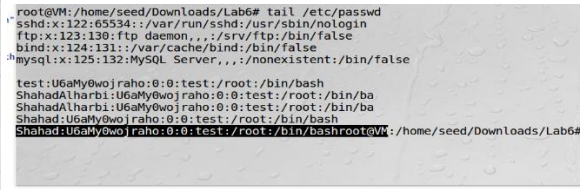
Chown root vulp: This allows the vulp executable to run with elevated privileges by changing its ownership to the root user.

With the Set-UID bit enabled, chmod 4755 vulp modifies the vulp executable's permissions to permit root-level execution.
To exploit the vulnerability in vulp.c, the attacker program attack.c is compiled into an executable called attack with the command gcc attack.c -o attack.

```
seed@ShahadAlharbi:~/.../Lab6$sudo sysctl -w fs.protected_symlinks=0
fs.protected_symlinks = 0
seed@ShahadAlharbi:~/.../Lab6$gcc vulp.c -o vulp
vulp.c: In function 'main':
vulp.c:20:42: warning: implicit declaration of function 'strlen' [-Wimplicit-fu
nction-declaration]
             fwrite(buffer, sizeof(char), strlen(buffer), fp);
                                          ^
vulp.c:20:42: warning: incompatible implicit declaration of built-in function '
strlen'
vulp.c:20:42: note: include '<string.h>' or provide a declaration of 'strlen'
seed@ShahadAlharbi:~/.../Lab6$sudo chown root vulp && sudo chmod 4755 vulp
seed@ShahadAlharbi:~/.../Lab6$gcc attack.c -o attack
seed@ShahadAlharbi:~/.../Lab6$
```

**Running the attack program:** Open a new window and Run the attacker program. The program will persistently seek to take advantage of the vulnerability present in the vulp progran by establishing symbolic links: /tmp/XYZ directed towards /dev/null and /tmp/ABC directed towards /etc/passwd. Subsequently, the program will engage in an infinite loop, continuously invoking the renameat2 system call in an effort to interchange these links at intervals of 10 milliseconds.




To find out whether the attacker program is successfully making symbolic links, by using the 'ls -l /tmp' command, which displays the contents of the '/tmp' directory as well as detailed file information. In the output, symbolic links are indicated by a 'l' at the start of the line, followed by permissions, ownership information, and the link's target. observe entries for /tmp/XYZ and /tmp/ABC, where /tmp/XYZ directs to /dev/null and /tmp/ABC directs to /etc/passwd. This output verifies that the attacker program is successfully handling the symbolic links required for executing the exploit against the susceptible vulp program.

Now run the target process but before we need to add execute privilege, after multiple attempt the attacks runs and gained root privilege